

Free Form Testing Agent

Project Component Map and Data Flow

Version: 1.0

Status: Living Architecture Guide

Last Updated: July 2026

1. Purpose of This Document

The Free Form Testing Agent has grown from a simple application explorer into a larger system containing:

- Application drivers
- State observation
- State graphs
- Action selection
- Exploration targets
- Coordinators
- Replay
- Persistence
- Sessions
- Interrupted session recovery
- Memory
- Failure handling
- Future planners and AI strategies

Because these components work together, it can become difficult to remember:

- What each component does.
- Why each component exists.
- Which component calls another component.
- What data moves between components.
- Which component modifies the data.
- Which data is temporary.
- Which data is persisted.
- How interrupted exploration resumes.

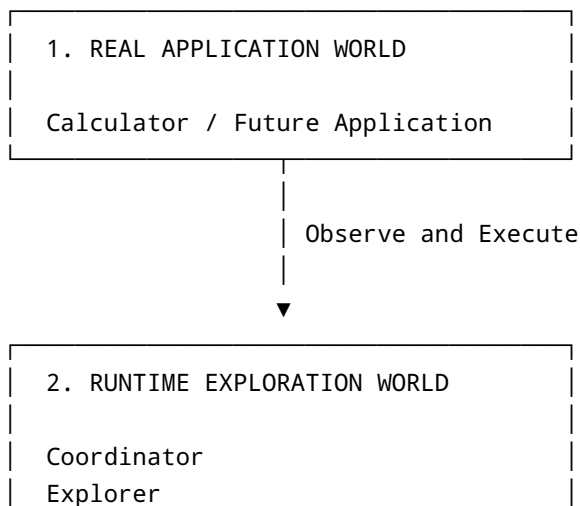
This document provides a simple mental model of the entire system.

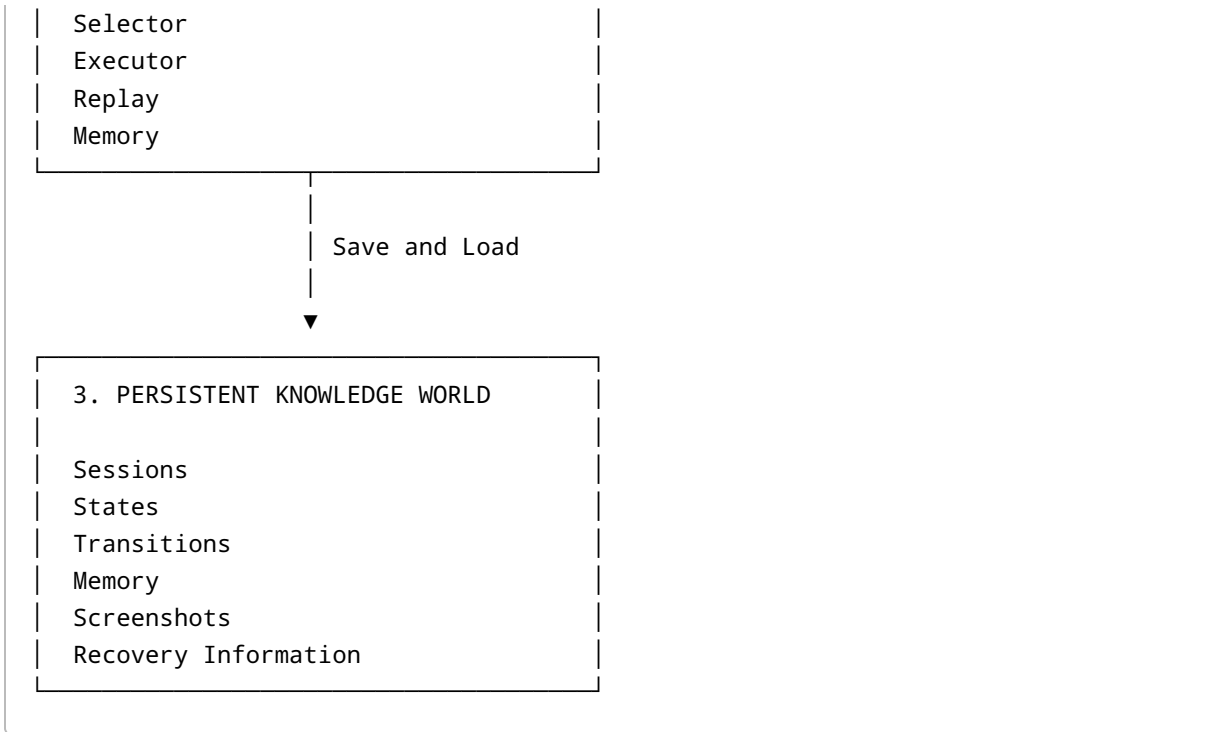
The shortest possible summary of the project is:

```
Observe Application
↓
Convert UI into State
↓
Store State in Graph
↓
Find Something Unexplored
↓
Restore Application to Required State
↓
Select an Action
↓
Execute the Action
↓
Observe the Result
↓
Create a Transition
↓
Update Graph and Memory
↓
Persist Everything
↓
Repeat
```

2. The Most Important Mental Model

The project has three major worlds.





The real application is the physical truth.

The runtime exploration system decides what to do.

The persistence system remembers what happened.

3. Core Data Objects

Before understanding the components, it is important to understand the main data objects moving through the system.

3.1 Application State

An Application State is a snapshot of the application.

Example:

Calculator Display: 7

Available Actions:

0

1
2
3
4
5
6
7
8
9
+
-
*
/
=

A simplified state object:

```
AppState(  
  id="state-123",  
  signature="abc456",  
  values={  
    "display": "7"  
  },  
  available_actions=[  
    Click("0"),  
    Click("1"),  
    Click("+"),  
    Click("=")  
  ]  
)
```

The state answers:

What does the application look like right now?

3.2 Action

An Action represents something the agent can do.

Examples:

```
Click 7
Click +
Click =
Enter Text
Press Enter
```

Example object:

```
Action(
  type="CLICK",
  target="num7Button"
)
```

The action answers:

```
What can the agent do?
```

3.3 Transition

A Transition records what happened when an action was executed.

```
Source State
  +
Action
  =
Target State
```

Example:

```
Display = 0
  +
Click 7
  =
Display = 7
```

Object:

```
Transition(
  source_state="S0",
```

```
    action="CLICK:num7Button",  
    target_state="S1"  
)
```

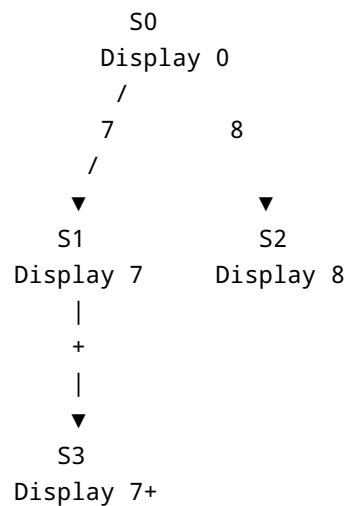
The transition answers:

How did the application move from one state to another?

3.4 State Graph

The State Graph stores all discovered application behavior.

Example:



The graph contains:

Nodes = States

Edges = Transitions

The graph answers:

What parts of the application have we discovered?

3.5 Exploration Memory

Exploration Memory stores operational exploration knowledge.

Example:

```
Visited States  
  
Explored Actions  
  
Unexplored Actions  
  
Current Path  
  
Pending Exploration Targets
```

The graph knows:

```
What application behavior exists?
```

The memory knows:

```
What have we already done, and what should we still do?
```

3.6 Exploration Target

An Exploration Target is a state that still contains unexplored work.

Example:

```
State S0  
  
Actions:  
  
7    → Explored  
8    → Explored  
9    → Not Explored  
+    → Not Explored
```

Therefore:

S0 is still an Exploration Target.

The target usually contains enough information to answer:

Which state should we return to?

Why should we return there?

What actions remain unexplored?

3.7 Exploration Session

A Session represents one complete testing run.

Example:

Session ID: session-123

Status: RUNNING

Started At: 10:00

States Discovered: 15

Transitions Discovered: 22

Explored Actions: 22

Possible statuses:

CREATED

RUNNING

INTERRUPTED

COMPLETED

FAILED

The session answers:

Which testing run does this data belong to?

4. Component Overview

The major project components are:

Mission / Entry Point

Session Manager

Recovery Manager

Coordinator

Explorer

Target Selector

Replay / State Restorer

Action Selector

Executor

Application Driver

State Observer

State Registry

Graph Manager

Exploration Memory

Persistence Layer

Screenshot Storage

Failure Detection

Planner

The following sections explain each component.

5. Entry Point / Runtime Bootstrap

Simple Description

The Entry Point starts the testing system.

It creates and connects all required components.

Main Responsibilities

Load Configuration

Create Driver

Create Graph

Create Memory

Create Repositories

Create Explorer

Create Replay Service

Create Coordinator

Start or Resume Session

Example

```
driver = CalculatorDriver()
```

```
graph = StateGraph()
```

```
memory = ExplorationMemory()

explorer = Explorer(
    driver=driver,
    graph=graph,
    memory=memory
)

coordinator = ExplorationCoordinator(
    explorer=explorer
)

coordinator.run()
```

Mental Model

Entry Point = Builds the machine.

It should not contain exploration logic.

6. Session Manager

Simple Description

The Session Manager manages the lifetime of one exploration run.

Main Responsibilities

- Create Session
- Start Session
- Mark Session Running
- Mark Session Interrupted
- Mark Session Completed
- Store Stop Reason

Load Existing Session

Example

Before exploration:

Session:

Status = CREATED

When exploration starts:

Status = RUNNING

If the process stops unexpectedly:

Status remains RUNNING on disk

During the next startup, recovery detects:

Persisted RUNNING session
+
No active previous runtime
=
Interrupted Session

The session can then be recovered.

Mental Model

Session Manager = Keeps track of the testing run.

7. Coordinator

Simple Description

The Coordinator is the main control loop.

It keeps the exploration process running.

Main Responsibilities

```
Start Exploration  
  
Ask for Next Exploration Target  
  
Restore Target State  
  
Run Exploration Step  
  
Check Stop Conditions  
  
Persist Progress  
  
Continue or Stop
```

Important Rule

The Coordinator should orchestrate components.

It should not contain the internal logic of those components.

Bad:

```
coordinator.calculate_state_hash()  
  
coordinator.choose_button()  
  
coordinator.click_ui_element()
```

Better:

```
state = observer.observe()

target = target_selector.select()

result = explorer.explore_step()

persistence.save(result)
```

Example

```
while session.is_running:

    target = target_selector.select_next_target()

    replay.restore(target)

    result = explorer.explore_step()

    persistence.save(result)

    if stop_policy.should_stop():
        break
```

Mental Model

Coordinator = The conductor of the orchestra.

It tells components when to work.

It does not perform their specialized jobs.

8. Explorer

Simple Description

The Explorer performs exploration work.

The most important unit is:

One Exploration Step

Main Responsibilities

Receive Current State

Get Available Actions

Ask Selector for One Action

Execute One Action

Observe Result

Create Transition

Update Graph

Update Memory

Return Step Result

Single Exploration Step

Source State
↓
Find Available Actions
↓
Select One Action
↓
Execute
↓
Observe Target State
↓
Create Transition
↓
Return Result

Example

Input:

```
State S0  
Display = 0
```

Selected action:

```
CLICK 7
```

Result:

```
State S1  
Display = 7
```

Transition:

```
S0 --CLICK(7)--> S1
```

Mental Model

```
Explorer = Performs one piece of exploration work.
```

9. Target Selector

Simple Description

The Target Selector decides which previously discovered state should be explored next.

This is different from the Action Selector.

Target Selector Question

Which state should we explore?

Action Selector Question

Which action should we execute from that state?

Example

Graph:

S0
7 → Explored
8 → Explored
9 → Unexplored

Another state:

S1
+ → Unexplored

The Target Selector chooses:

S0

or:

S1

depending on the exploration strategy.

BFS Example

Breadth-first target selection prefers shallower states.

Depth 0: S0

Depth 1: S1, S2

Depth 2: S3, S4

The BFS selector explores unfinished shallow states first.

Mental Model

Target Selector = Decides where to go.

10. Action Selector

Simple Description

The Action Selector chooses one action from the current state.

Input

Current State

Available Actions

Exploration Memory

Graph Knowledge

Output

One Action

or:

No Action

Example

Available:

CLICK 7

CLICK 8

CLICK 9

Memory:

7 = Explored

8 = Explored

9 = Unexplored

Selected:

CLICK 9

Deterministic Rule

The same input should produce the same output.

Same State

+

Same Available Actions

+

Same Memory

=

Same Selected Action

Mental Model

Action Selector = Decides what to do.

11. Executor

Simple Description

The Executor performs an Action.

It converts a generic framework action into a real application operation.

Input

```
Action(  
    type="CLICK",  
    target="num7Button"  
)
```

Execution

Find Control

Verify Control

Click Control

Capture Execution Result

Output

Success

or:

Execution Failure

Important Rule

The Executor does not decide which action should be performed.

It only performs the action it receives.

Mental Model

Executor = The hands of the agent.

12. Application Driver

Simple Description

The Application Driver knows how to communicate with the real application.

For the current milestone, this is the Calculator Driver.

Main Responsibilities

```
Launch Application  
Close Application  
Find Window  
Find Controls  
Click Controls  
Read Values  
Check Process  
Capture Screenshot
```

Example

Framework action:

```
CLICK:num7Button
```

Driver operation:

```
Find Windows UI Automation control:  
  
AutomationId = num7Button  
  
Invoke Click
```

Future Drivers

```
Calculator Driver  
  
Generic Windows Driver  
  
Web Driver
```

Mobile Driver

API Driver

Mental Model

Driver = Translator between the framework and the real application.

13. State Observer

Simple Description

The State Observer looks at the real application and creates structured state data.

Input

Real Running Application

Processing

Read Window

Read Controls

Read Display

Discover Available Actions

Normalize Data

Create State Signature

Output

```
ApplicationState(...)
```

Example

Real application:

```
Calculator display shows:
```

```
7
```

Observer output:

```
ApplicationState(  
  values={  
    "display": "7"  
  }  
)
```

Mental Model

```
Observer = The eyes of the agent.
```

14. State Signature / State Identity

Simple Description

The State Signature determines whether the current application state has already been seen.

Input

Raw observed state:

```
Display = 7
```

```
Buttons = [...]
```

Processing

```
Normalize Meaningful Data
```

```
Remove Unstable Data
```

```
Serialize
```

```
Hash
```

Output

```
State Signature:
```

```
abc123
```

Why It Matters

Without state identity:

```
Display 7
```

could be stored 100 times as 100 different states.

With state identity:

```
Observed State
```

```
↓
```

```
Generate Signature
```

```
↓
```

```
Find Existing Signature
```

↓
Reuse Existing State

Mental Model

State Signature = Fingerprint of application behavior.

15. State Registry

Simple Description

The State Registry decides whether an observed state is new or already known.

Input

Observed Application State

Processing

Check State Signature

Output A: New State

Create State ID

Add to Graph

Persist State

Output B: Existing State

Return Existing State

Example

Observed:

Display = 7

Registry lookup:

Signature abc123 already exists.

Result:

Reuse State S5.

Mental Model

State Registry = Prevents duplicate states.

16. Graph Manager

Simple Description

The Graph Manager owns the discovered model of application behavior.

Main Responsibilities

Add States

Add Transitions

Find States

Find Paths

Find Unexplored Actions

Calculate Coverage

Support Replay Path Lookup

Example

Before action:

S0

After:

S0 --CLICK(7)--> S1

The Graph Manager stores:

Node:

S0

Node:

S1

Edge:

S0 → S1

Mental Model

Graph Manager = Map of the discovered application.

17. Exploration Memory

Simple Description

Exploration Memory stores knowledge about the exploration process.

Typical Data

Visited States

Explored State-Action Pairs

Pending Targets

Failed Actions

Current Path

Exploration Counters

Example

State S0:

CLICK 7 = Explored

CLICK 8 = Explored

CLICK 9 = Unexplored

The Action Selector uses this information to choose:

CLICK 9

Difference Between Graph and Memory

Graph:

What behavior was discovered?

Memory:

What exploration work has been completed?

Mental Model

Memory = The agent's exploration notebook.

18. Replay / State Restoration

Simple Description

Replay returns the real application to a previously discovered state.

This is necessary because the graph contains many states, but the real application can physically be in only one state.

Example

Graph:

S0
/
A B

↓ ↓
S1 S2

The agent explores:

S0 --A--> S1

The real application is now at:

S1

To explore action B from S0, the agent must:

Restart Application

Replay Path to S0

Verify S0

Execute B

Replay Flow

```
Target State
  ↓
Find Known Path
  ↓
Reset Application
  ↓
Start from Initial State
  ↓
Execute Path Actions
  ↓
Observe Each Result
  ↓
Verify Expected States
  ↓
Reach Target
```

Mental Model

Replay = Travel back to a known place in the application.

19. Replay Verification

Simple Description

Replay Verification ensures the application actually reached the expected state.

Example

Expected:

State S5
Display = 7+

Actual:

Display = 8

Result:

Replay Mismatch

Why It Matters

Without verification:

Replay Action Failed
↓
Framework Continues Anyway
↓
Exploration Starts from Wrong State

↓
Graph Becomes Corrupted

With verification:

Replay Action
↓
Observe State
↓
Compare Expected vs Actual
↓
Continue Only if Correct

Mental Model

Replay Verification = Confirms we arrived at the correct destination.

20. Persistence Layer

Simple Description

The Persistence Layer saves runtime knowledge to disk.

Without persistence:

Process Stops
↓
Everything Is Lost

With persistence:

Process Stops
↓
Graph Remains

Memory Remains

Session Remains

Screenshots Remain

Data Saved

Sessions

States

Transitions

Exploration Memory

Screenshots

Recovery Metadata

Important Rule

Runtime objects should not directly write files or database records everywhere.

Preferred architecture:

Runtime Component

↓

Repository / Persistence Interface

↓

Disk Storage

Mental Model

Persistence = Long-term memory of the framework.

21. Graph Persistence

Simple Description

Graph Persistence saves the discovered application map.

Before Save

Runtime:

```
S0 --7--> S1 --+--> S2
```

On Disk

Stored:

States:

```
S0  
S1  
S2
```

Transitions:

```
S0 → S1  
S1 → S2
```

After Restart

Load States

Load Transitions

Rebuild Graph

Result:

```
S0 --7--> S1 --+--> S2
```

22. Memory Persistence

Simple Description

Memory Persistence saves exploration progress.

Example

Before interruption:

```
Explored Actions = 14  
Unexplored Actions = 50
```

After restart:

```
Load Memory
```

Result:

```
Explored Actions = 14  
Continue from Action 15
```

Without memory persistence, the graph may survive but the framework may not know what work was already completed.

23. Screenshot Storage

Simple Description

Screenshot Storage saves visual evidence.

Screenshots Can Belong To

```
States
Failures
Replay Mismatches
Sessions
Transitions
```

Example

```
Session
├──
│   └── State S5
│       └── screenshot.png
```

Screenshots are usually stored as files.

The database or metadata stores the path.

Mental Model

```
Screenshot Storage = Visual evidence archive.
```

24. Interrupted Session Recovery

Simple Description

Interrupted Session Recovery continues an exploration session after the previous runtime stopped unexpectedly.

This is the milestone that has just been completed.

Normal Completion

```
Session RUNNING
  ↓
Exploration Finishes
  ↓
Session COMPLETED
```

Interrupted Execution

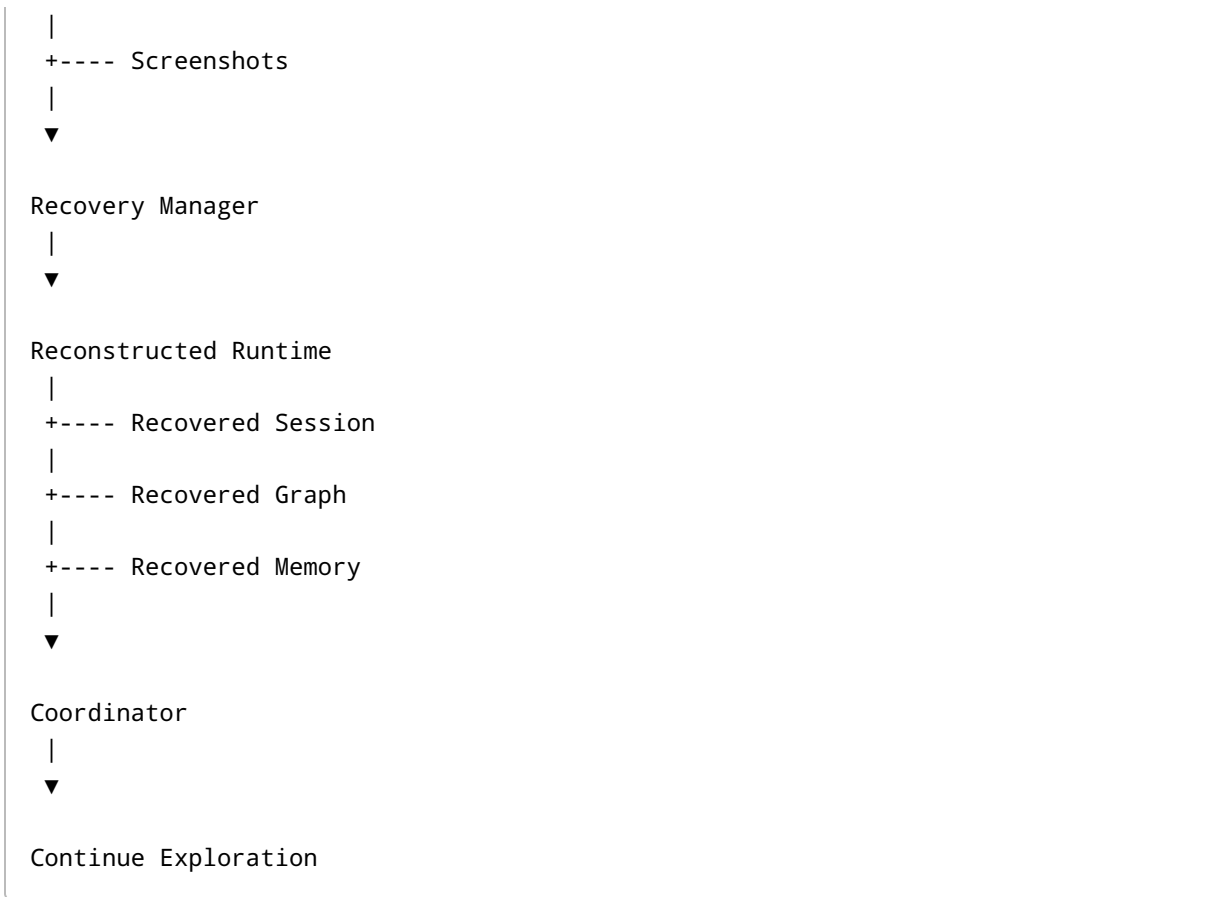
```
Session RUNNING
  ↓
Process Stops Unexpectedly
  ↓
Session Remains RUNNING on Disk
```

On the next runtime:

```
Load Session
  ↓
Detect Previous RUNNING State
  ↓
Mark as Interrupted / Recoverable
  ↓
Load Graph
  ↓
Load Memory
  ↓
Continue Exploration
```

Recovery Data Flow

```
Disk
|
+---- Session
|
+---- Graph
|
+---- Memory
```



Recovery Success Condition

Before interruption:

Transitions = 14

After recovery:

Transitions = 14

After continued exploration:

Transitions = 16

The key requirement is:

```
Recovered Runtime State
=
Persisted State Before Interruption
```

Mental Model

```
Recovery = Rebuild the agent's brain after the process dies.
```

25. Recovery Manager

Simple Description

The Recovery Manager reconstructs runtime components from persisted data.

Main Responsibilities

```
Detect Interrupted Session

Load Session

Load Graph

Load Memory

Validate Loaded Data

Reconstruct Runtime

Return Recovery Result
```

Example

```
recovery = recovery_manager.recover(session_id)

coordinator.run(
```

```
    session=recovery.session,  
    graph=recovery.graph,  
    memory=recovery.memory  
)
```

Important Rule

Recovery should restore knowledge.

Replay restores the physical application.

These are different operations.

Recovery:

Disk → Runtime Knowledge

Replay:

Graph Knowledge → Real Application State

Mental Model

Recovery Manager = Restores the framework.

Replay = Restores the application.

This distinction is extremely important.

26. Failure Detector

Simple Description

The Failure Detector identifies deterministic failures.

Initial Failure Types

Application Crash
Application Hang
Window Disappearance
Execution Failure
Observation Failure
Replay Mismatch
Unexpected Error Dialog

Example

Before action:

Process Alive = True

Execute:

CLICK =

After action:

Process Alive = False

Result:

Crash Failure

Mental Model

Failure Detector = Detects that something broke.

AI should not be required to determine whether the process crashed.

27. Validator

Simple Description

A Validator determines whether behavior is correct.

Failure Detection asks:

Did something break?

Validation asks:

Was the result correct?

Example

Input:

$2 + 3$

Expected:

5

Actual:

6

Result:

Validation Failure

Future Validator Types

Deterministic Validator

Arithmetic Validator

Requirement Validator

Semantic Validator

AI Validator

Mental Model

Validator = Judges correctness.

28. Planner

Simple Description

The Planner is a higher-level future component.

It decides what testing objective should be pursued.

Selector vs Planner

Action Selector:

Which button should I click next?

Planner:

What should I try to test?

Example

Planner goal:

Test division by zero.

Plan:

Enter non-zero number

Press divide

Enter zero

Press equals

The deterministic framework converts the plan into real executable actions.

Mental Model

Planner = Decides what the agent is trying to achieve.

29. Requirement Model

Simple Description

The Requirement Model represents expected product behavior.

Contains

Goals

Preconditions

Workflows

Expected Outcomes

Coverage

Example

Requirement:

The calculator should correctly add two positive numbers.

Structured form:

Goal:

Perform addition.

Precondition:

Calculator is ready.

Workflow:

Enter first number.

Press plus.

Enter second number.

Press equals.

Expected Outcome:

Correct sum is displayed.

Mental Model

Requirement Model = What the application is supposed to do.

30. AI Strategy

Simple Description

AI improves decision-making but does not replace deterministic infrastructure.

Initial model:

```
qwen2.5-coder:7b
```

```
through Ollama
```

AI Can Help With

```
Target Ranking
```

```
Action Selection
```

```
Workflow Planning
```

```
Requirement Interpretation
```

```
Failure Analysis
```

AI Should Not Own

```
Application Execution
```

```
State Persistence
```

```
Crash Detection
```

```
Graph Integrity
```

```
Replay Verification
```

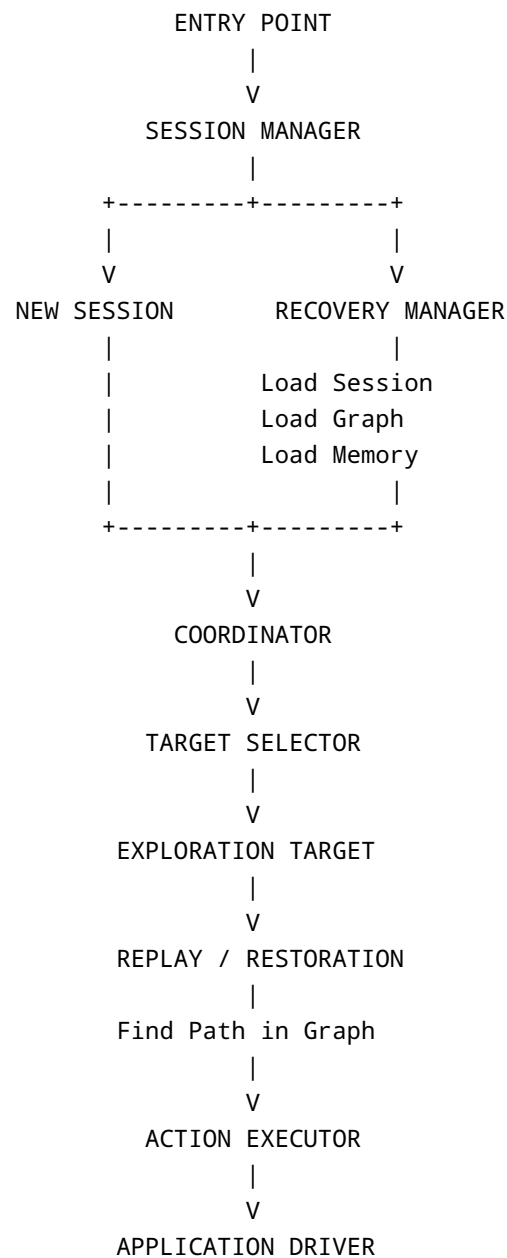
```
Session Recovery
```

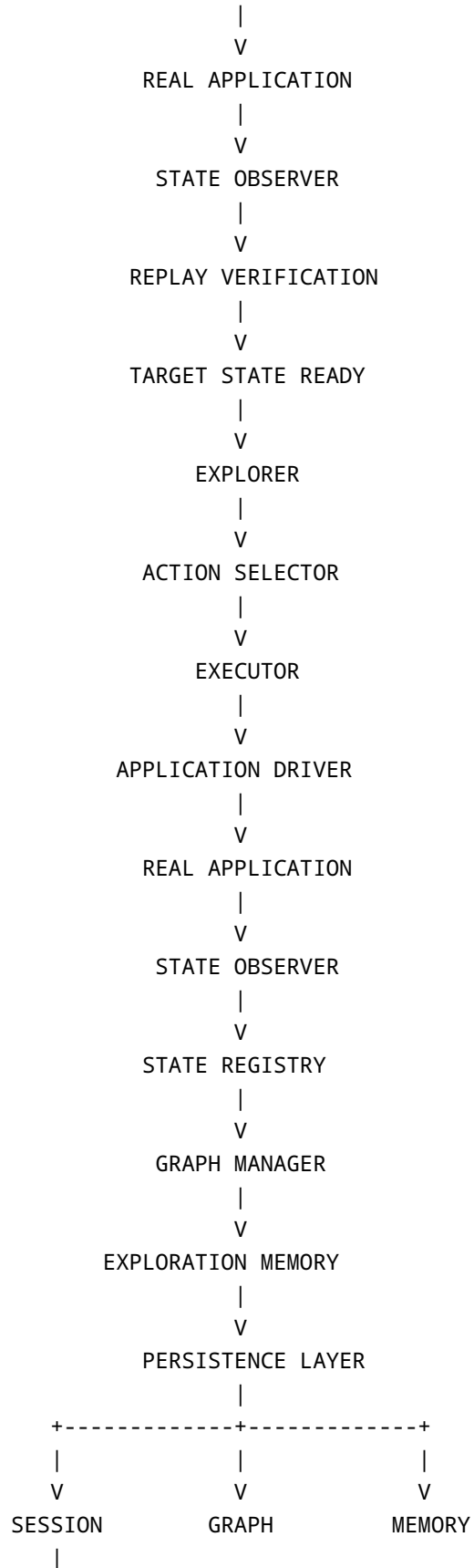
Mental Model

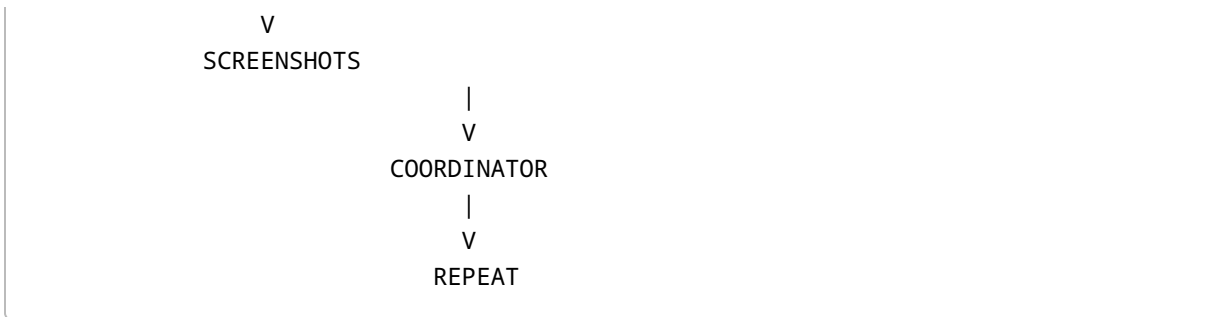
AI = Strategy advisor.

Deterministic framework = Source of truth.

31. Complete Component Relationship Map

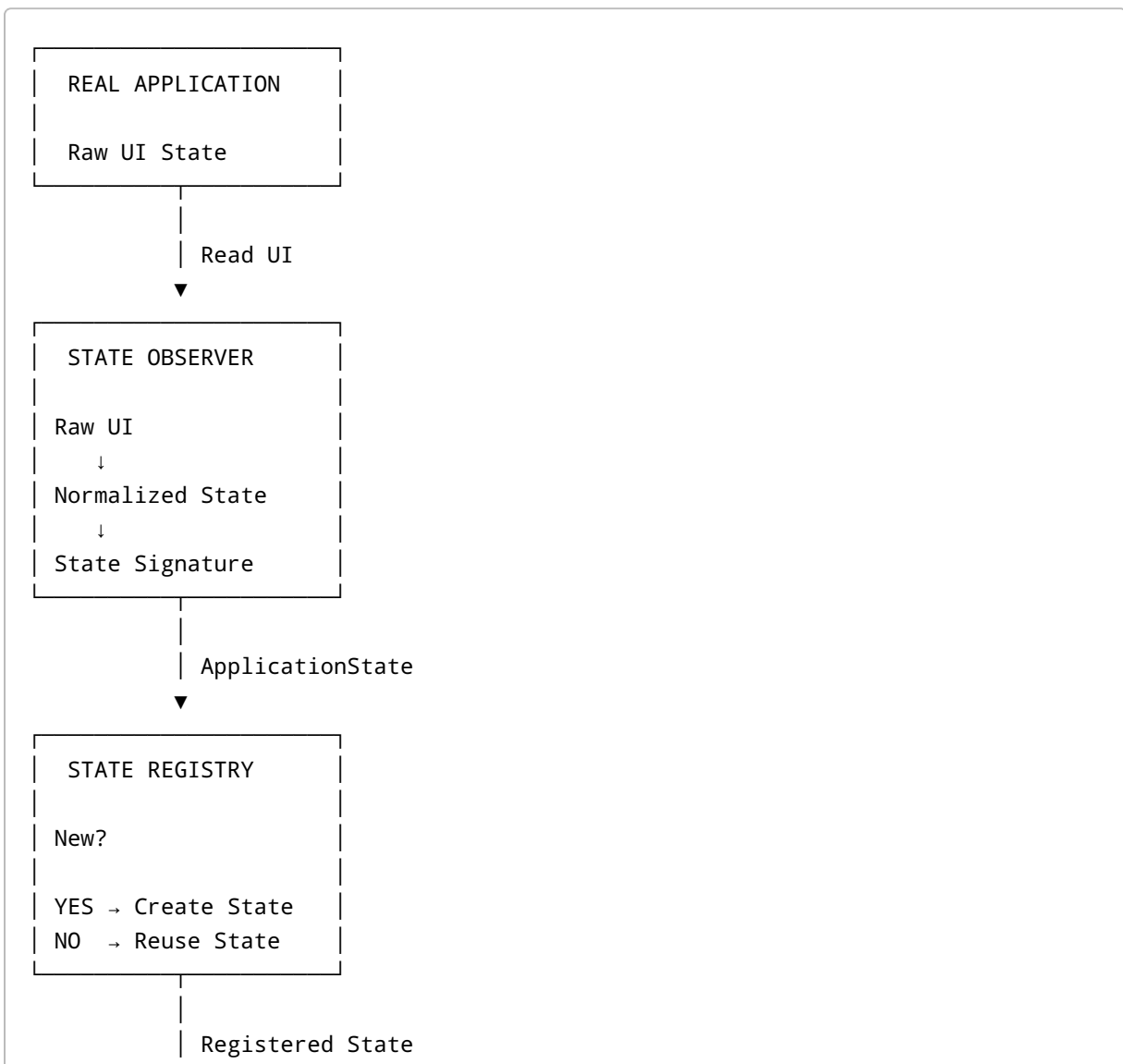


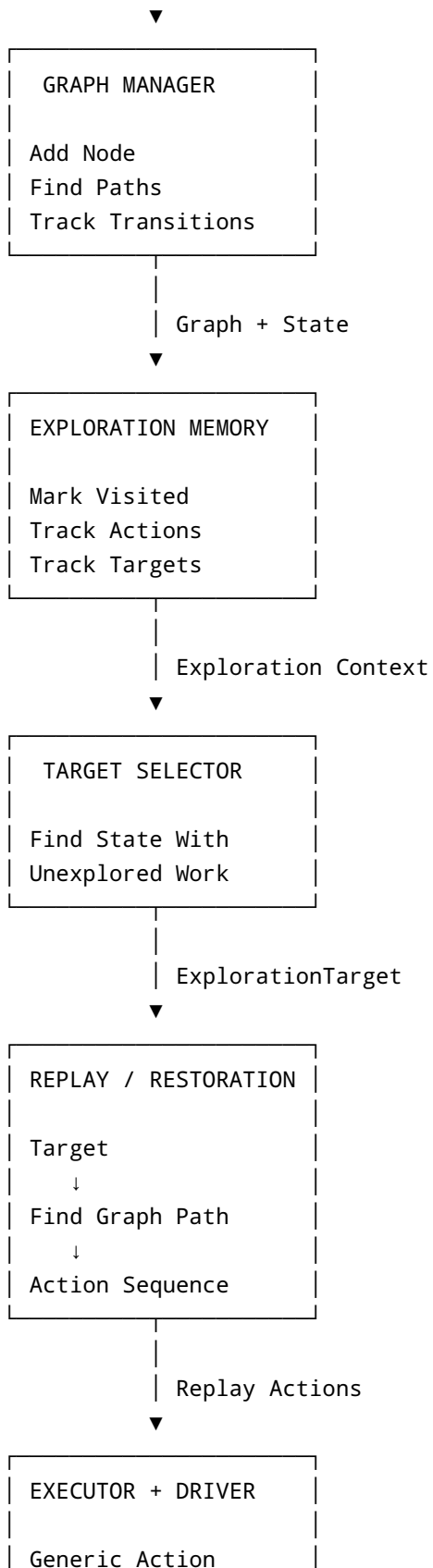


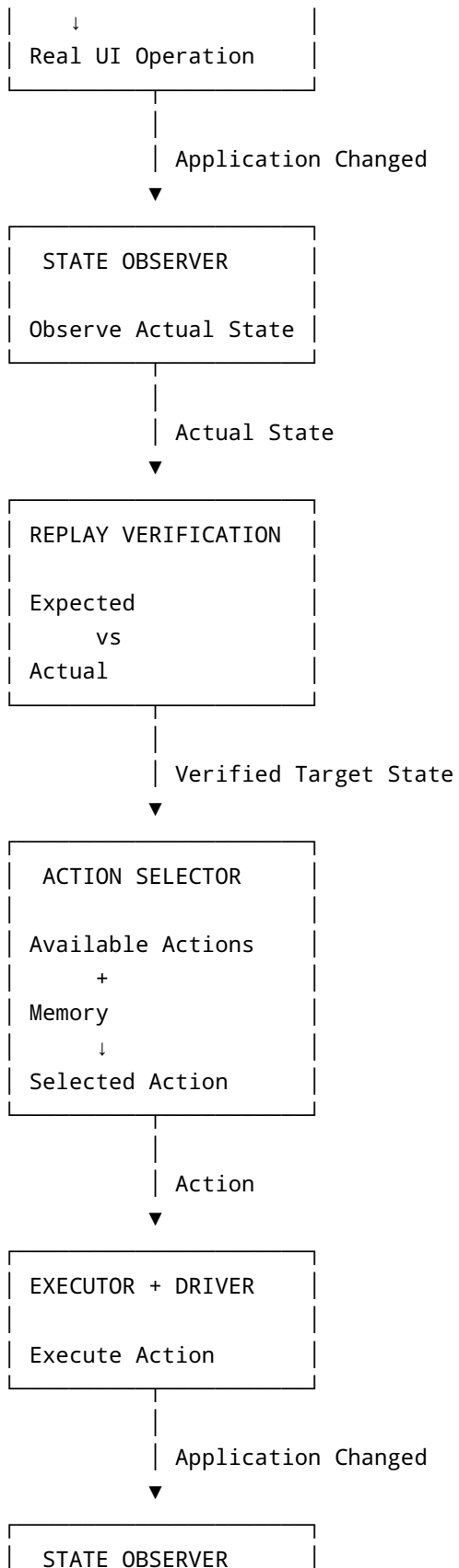


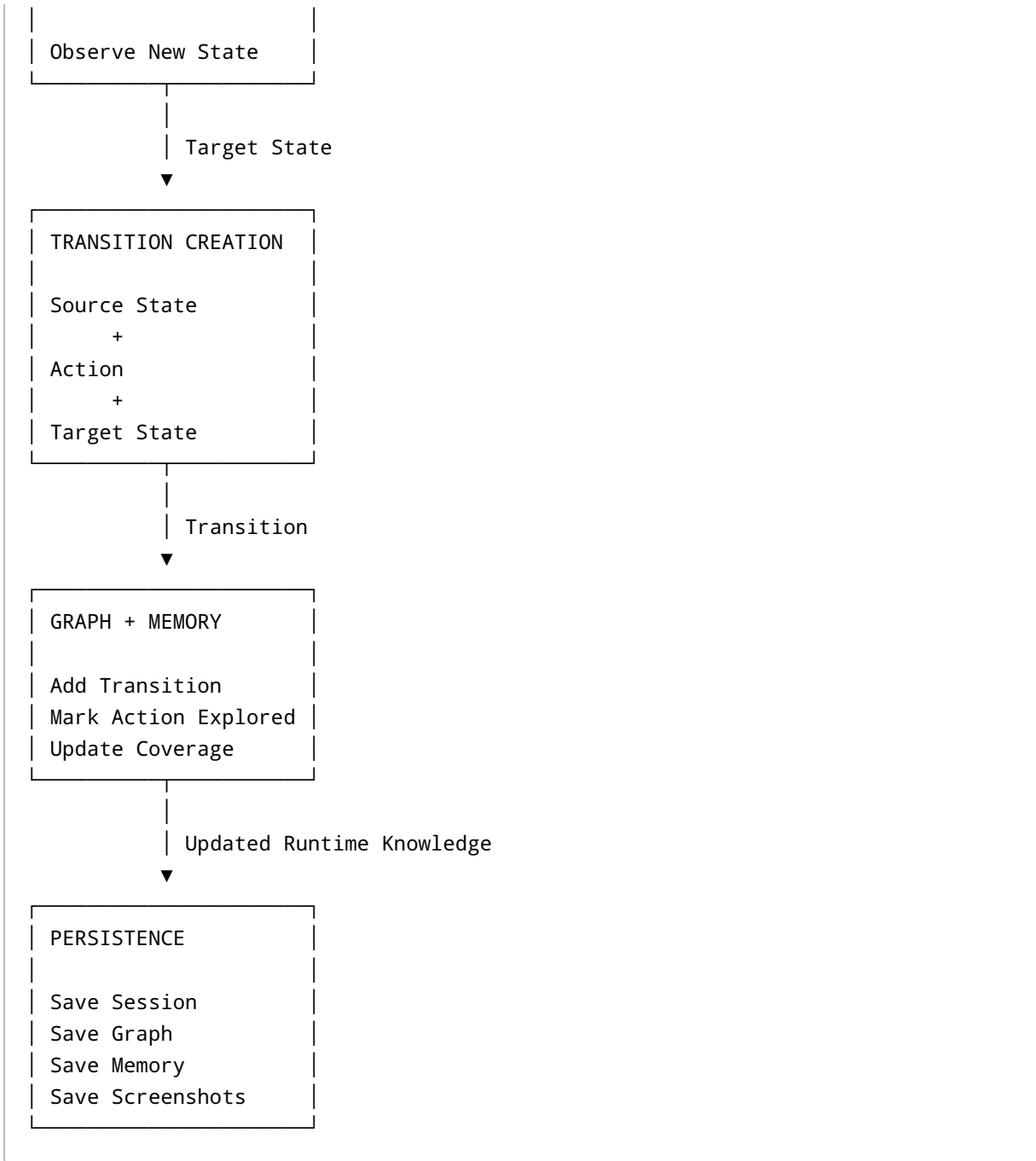
32. Main Data Flow Diagram

The following diagram focuses on the data itself.









33. Data Transformation Table

Component	Receives	Changes	Produces
Driver	Framework command	Converts command to app-specific operation	Real application interaction

Component	Receives	Changes	Produces
Observer	Raw application UI	Normalizes UI data	Application State
State Signature	State data	Removes unstable values and hashes meaningful data	Stable state identity
State Registry	Observed state	Deduplicates state	New or existing registered state
Graph Manager	States and actions	Creates nodes and transitions	Application behavior graph
Memory	Exploration results	Marks visited and explored work	Updated exploration knowledge
Target Selector	Graph and memory	Ranks or filters unfinished states	Exploration Target
Replay	Target and graph	Converts target into action path	Replay action sequence
Replay Verification	Expected and actual state	Compares identities	Verified state or mismatch
Action Selector	State, actions, memory	Filters explored actions	Selected action
Executor	Generic action	Executes through driver	Execution result
Explorer	Source, action, result	Builds exploration transaction	Step result and transition
Coordinator	Runtime results	Controls execution lifecycle	Continued or stopped exploration
Persistence	Runtime objects	Serializes data	Durable disk state
Recovery Manager	Persisted data	Reconstructs runtime objects	Recovered runtime
Failure Detector	Execution evidence	Applies deterministic rules	Failure record
Validator	Expected and actual behavior	Compares correctness	Validation result
Planner	Goal and knowledge	Creates testing strategy	Workflow or plan
AI Strategy	Structured testing context	Ranks or interprets options	Strategic recommendation

34. Complete Normal Exploration Data Flow

1. Application exists physically.
2. Observer reads the application.
3. Raw UI becomes ApplicationState.
4. State signature identifies the state.
5. Registry deduplicates the state.
6. Graph stores the state.
7. Memory records exploration knowledge.
8. Target Selector finds unfinished work.
9. Replay restores the physical application.
10. Replay Verification confirms the correct state.
11. Action Selector chooses one unexplored action.
12. Executor sends the action to the Driver.
13. Driver changes the real application.
14. Observer reads the changed application.
15. Registry identifies the resulting state.
16. Transition connects source and target states.
17. Graph is updated.
18. Memory marks the state-action pair explored.
19. Persistence saves runtime knowledge.
20. Coordinator starts the next iteration.

35. Interrupted Recovery Data Flow

NORMAL RUNTIME

Coordinator



Graph Changes



Memory Changes



Session Changes



Persist to Disk



PROCESS INTERRUPTED

Later:

NEW RUNTIME



Load Persisted Session



Detect Interrupted Session



Load Graph



Load Memory



Reconstruct Runtime



Compare Runtime with Disk



Resume Coordinator



Select Next Unexplored Target



Replay to Target



Continue Exploration

The important idea is:

Persistence preserves knowledge.

Recovery reconstructs knowledge.

Replay reconstructs physical application state.

Coordinator continues work.

36. Component Responsibilities in One Sentence

Entry Point

→ Builds the system.

Session Manager

→ Tracks the testing run.

Coordinator

→ Controls the overall loop.

Explorer

→ Performs one exploration transaction.

Target Selector

→ Chooses where to explore.

Replay

→ Returns the app to a known state.

Replay Verification

→ Confirms restoration succeeded.

Action Selector

→ Chooses what to do.

Executor

→ Performs the selected action.

Driver

→ Communicates with the real application.

Observer

→ Converts the real UI into structured data.

State Registry

→ Prevents duplicate states.

Graph Manager

→ Stores discovered application behavior.

Memory

→ Tracks completed and remaining exploration work.

Persistence

→ Saves runtime knowledge.

Recovery Manager

→ Rebuilds runtime knowledge after interruption.

Failure Detector

→ Detects deterministic failures.

Validator

→ Determines whether behavior is correct.

Planner

→ Decides what testing objective to pursue.

Requirement Model

→ Represents what the application should do.

AI Strategy

→ Improves testing decisions.

37. The Project in Four Loops

The easiest way to remember the entire project is as four loops.

Loop 1: Observation Loop

Application

↓

Observer

↓

State

↓

Registry

↓
Graph

Purpose:

Understand the application.

Loop 2: Exploration Loop

Graph + Memory
↓
Target Selector
↓
Replay
↓
Action Selector
↓
Executor
↓
New State

Purpose:

Discover new behavior.

Loop 3: Persistence Loop

Runtime Graph
Runtime Memory
Runtime Session
↓
Persistence
↓
Disk

Purpose:

Prevent knowledge loss.

Loop 4: Recovery Loop

```
Disk
  ↓
Recovery Manager
  ↓
Runtime Graph
Runtime Memory
Runtime Session
  ↓
Replay
  ↓
Physical Application State
  ↓
Continue Exploration
```

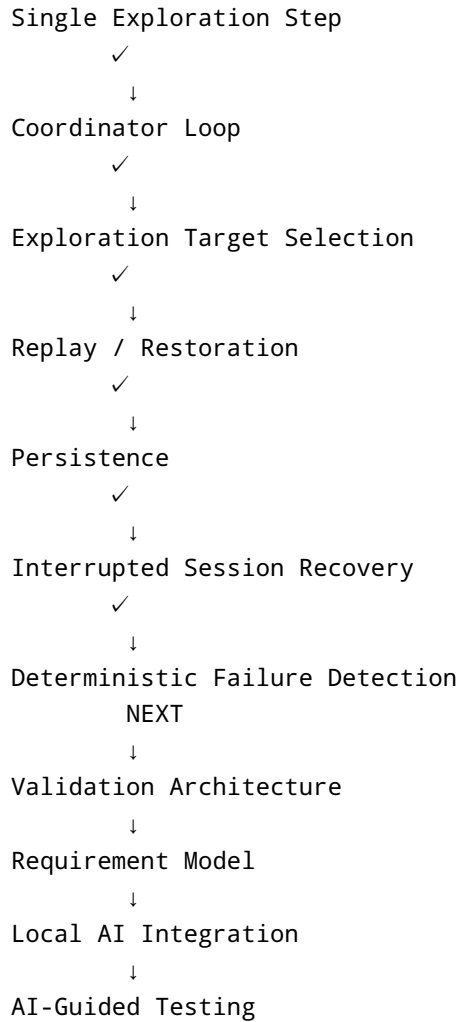
Purpose:

Continue after interruption.

38. Current Project Position

The project has now reached this point:

```
Application Interaction
  ✓
  ↓
State Representation
  ✓
  ↓
State Graph
  ✓
  ↓
Deterministic Action Selection
  ✓
  ↓
```



39. Final Mental Map

When the project feels too large, remember only this:

OBSERVER
sees the application.

STATE REGISTRY
identifies what was seen.

GRAPH
remembers where we have been.

MEMORY

remembers what we have done.

TARGET SELECTOR
chooses where to go.

REPLAY
takes us there.

ACTION SELECTOR
chooses what to do.

EXECUTOR
does it.

OBSERVER
sees what happened.

EXPLORER
turns that into a transition.

COORDINATOR
keeps the process running.

PERSISTENCE
saves everything.

RECOVERY
rebuilds everything after interruption.

FAILURE DETECTOR
finds what broke.

VALIDATOR
checks what is correct.

PLANNER
decides what should be tested.

AI
helps make better strategic decisions.

The core of the entire Free Form Testing Agent is still:

State
+
Action

=
Next State

Everything else exists to make that loop:

Deterministic

Explorable

Restorable

Persistent

Recoverable

Validatable

Eventually Intelligent